

STACK REVISION QNS

1. What is a stack in C++?

- a) A type of array
- b) A linear data structure following FIFO
- c) A linear data structure following LIFO
- d) A hierarchical data structure

Answer: c) A linear data structure following LIFO

2. What is the primary operation of a stack?

- a) Insert at the front and remove from the end
- b) Insert at the end and remove from the front
- c) Insert and remove from the same end
- d) Insert at any position and remove from any position

Answer: c) Insert and remove from the same end

3. Which of the following is NOT a standard stack operation?

- a) push()
- b) pop()
- c) insert()
- d) top()

Answer: c) insert()

4. What does the pop() function do in a stack?

- a) Inserts an element into the stack
- b) Removes and returns the top element
- c) Removes all elements from the stack
- d) Returns the top element without removing it

Answer: b) Removes and returns the top element

5. If an empty stack is popped, what will happen?

- a) The operation will be ignored
- b) The program will crash or throw an error
- c) It will return a default value
- d) It will push a new element

Answer: b) The program will crash or throw an error

6. What is the time complexity of push() and pop() operations in a stack (assuming an efficient implementation)?

- a) $O(1)$
- b) $O(n)$
- c) $O(\log n)$
- d) $O(n^2)$

Answer: a) $O(1)$

7. Which header file is required to use the standard stack in C++?

Answer: `<stack>`

8. Which of the following applications use a stack?

- a) Function call recursion
- b) Undo/Redo functionality
- c) Parenthesis balancing
- d) All of the above

Answer: d) All of the above

9. What is a common real-world example of a stack?

- a) Train coaches
- b) A queue at a ticket counter
- c) Plates stacked in a cafeteria
- d) Books arranged in a shelf

Answer: c) Plates stacked in a cafeteria

10. In function call management, the stack is used to store:

- a) Local variables
- b) Return addresses
- c) Function parameters
- d) All of the above

Answer: d) All of the above

11. How can a stack be implemented?

- a) Using an array
- b) Using a linked list
- c) Both a and b
- d) None of the above

Answer: c) Both a and b

12. What is the advantage of implementing a stack using a linked list over an array?

- a) Dynamic size allocation
- b) Fast element access
- c) Less memory usage
- d) Easier implementation

Answer: a) Dynamic size allocation

13. What happens if a stack implemented using an array exceeds its allocated memory?

- a) Stack Overflow
- b) Segmentation fault
- c) Memory Leak
- d) Stack Underflow

Answer: a) Stack Overflow

14. What is the time complexity of checking whether a stack is empty?

- a) $O(1)$
- b) $O(\log n)$
- c) $O(n)$
- d) $O(n^2)$

Answer: a) $O(1)$

15. What is the primary disadvantage of using a stack?

- a) It does not support random access
- b) Operations are slow
- c) It consumes too much memory
- d) It cannot be used for recursion

Answer: a) It does not support random access

16. What is the best way to reverse a string using a stack?

- a) Push each character onto the stack, then pop them one by one
- b) Swap characters manually
- c) Use a recursive function
- d) Use an additional array

Answer: a) Push each character onto the stack, then pop them one by one

17. What is the time complexity of checking whether a stack is empty?

- a) $O(1)$
- b) $O(\log n)$
- c) $O(n)$
- d) $O(n^2)$

Answer: a) $O(1)$

18. What is the primary disadvantage of using a stack?

- a) It does not support random access
- b) Operations are slow
- c) It consumes too much memory
- d) It cannot be used for recursion

Answer: a) It does not support random access

19. What is the best way to reverse a string using a stack?

- a) Push each character onto the stack, then pop them one by one
- b) Swap characters manually
- c) Use a recursive function
- d) Use an additional array

Answer: a) Push each character onto the stack, then pop them one by one